

Introduction to Compilers

1. What do you understand by the terms translator and compiler?

Ans: A **translator** or **language processor** is a program that translates an input program written in a programming language into an equivalent program in another language. **Compiler** is a type of translator, which takes a program written in a high-level programming language as input and translates into an equivalent program in low-level language such as machine language or assembly language. The program written in high-level language is known as **source program**, and the program converted into low-level language is known as **object** (or **target**) **program**. Moreover, compiler traces the errors in the source program and generates the error report. Without compilation, no program written in a high-level language can be executed. After compilation only the program in machine language is loaded into the memory for execution. For every programming language, we have a different compiler; however, the basic tasks performed by every compiler are same.

2. Explain the steps required for the execution of a high-level language program with the help of compiler.

Ans: The execution of a high-level language program is performed basically in two steps:

- ❑ **Compilation or translation:** During compilation the source program is translated into the target program. The target program can either be a machine code or an assembly language code. If the target program is executable machine language code, then it can be executed directly to generate the output. Figure 1.1 shows the compilation phase.

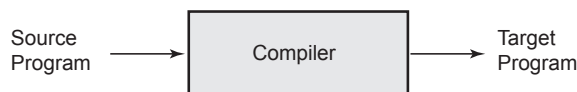


Figure 1.1 Compilation of Source Program

- ❑ **Execution of the target program:** During execution, the target program is first loaded into the main memory and then the user interacts with the target program to generate the output. The execution phase is shown in Figure 1.2.

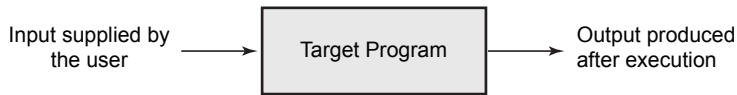


Figure 1.2 Executing Target Program

3. What are the difference between compiler and interpreter?

Ans: Compiler translates the whole source program into the target program in one step (see Figure 1.1). That is, it first scans the entire input program and then translates it into the target program. The target program is then executed separately for generating the output according to the given inputs.

Interpreter, on the other hand, directly executes the source program line by line according to the given inputs. That is, translation and execution of each statement are carried out side by side. Thus, separate execution of the program is not required. The line by line execution of the program provides better debugging environment than a compiler. The main drawback of an interpreter is that the execution time of an interpreted program is generally slower than that of a compiled program because the program needs to be translated every time it is executed. The interpretation process is shown in Figure 1.3.



Figure 1.3 Working of an Interpreter

4. What do you understand by the term cousins of compiler?

Ans: The term ‘cousins of compiler’ refers to the type of programs which are required for the execution of the source program. These are the programs along with which compiler operates. The cousins of compilers are *preprocessors*, *assemblers*, and *loaders and link editors*.

- **Preprocessors:** Before compilation, the source program is processed by the preprocessor to prepare it for the compilation. The preprocessor program creates modified source program from the original source program by replacing the preprocessor directives with the suitable content. The new source program acts as an input to the compiler (see Figure 1.4). Preprocessor performs various tasks as given here.
 - It permits the user to include the header files in the program and user can make use of the functions defined in these header files.
 - It permits the user to include macros in the program. **Macros** are the small set of instructions that are used in a program repetitively. Macros have two attributes, namely, *macro name* and *macro definition*. Whenever the macro name is encountered in the program then it is replaced by the macro definition (set of statements correspond to the macro).

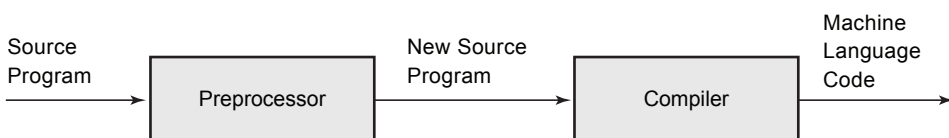


Figure 1.4 Preprocessor's Role

- ❑ **Assemblers:** In some cases, compiler generates the target program in assembly language. In that case, the assembly language program is given to the assembler as input. An assembler then translates the assembly language program into machine language program which is relocatable machine code. An assembly language program is in mnemonics.

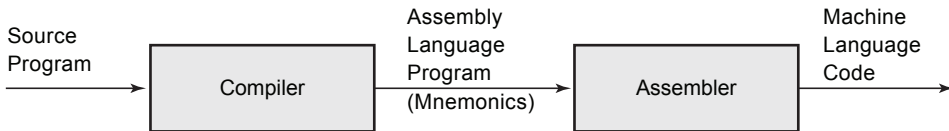


Figure 1.5 Assembler's Role

- ❑ **Loaders and link editors:** The larger source programs are compiled in small pieces by the compiler. To run the target machine code of any source program successfully, there is a need to link the relocated machine language code with library files and other relocatable object files. So, loader and link editor programs are used for the link editing and loading of the relocated codes. **Link editors** create a single program from several files of relocated machine code. **Loaders** read the relocated machine code and alter the relocatable addresses. To run the machine language program, the code with altered data and commands is placed at the correct location in the memory.

5. Discuss the steps involved in the analysis of a source program with the help of a block diagram.

Ans: The steps involved in the analysis of source program are given below.

- ❑ Source program acts as an input to the preprocessor. Preprocessor modifies the source code by replacing the header files with the suitable content. Output (modified source program) of the preprocessor acts as an input for the compiler.
- ❑ Compiler translates the modified source program of high-level language into the target program. If the target program is in machine language, then it can be executed directly. If the target program is in assembly language, then that code is given to the assembler for translation. Assembler translates the assembly language code into the relocatable machine language code.
- ❑ Relocatable machine language code acts as an input for the linker and loader. Linker links the relocatable code with the library files and the relocatable objects, and loader loads the integrated code into memory for the execution. The output of the linker and loader is the equivalent machine language code for the source code.

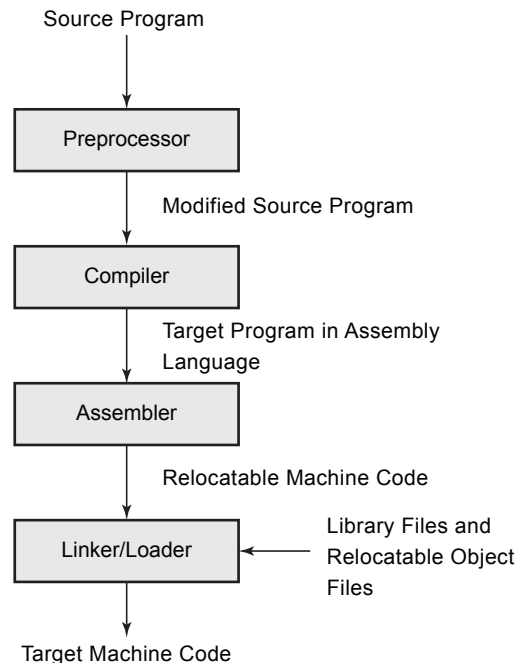


Figure 1.6 Block Diagram of Source Program Analysis

6. Explain the different phases of compiler with diagram.

Or

Explain the structure of compiler.

Ans: Compiler translates an input source program written in any high-level programming language into an equivalent target program in machine language. As compilation is a complex process, it is divided into several phases. A **phase** is a reasonably interrelated procedure that takes input in one representation and produces the output in another representation. The structure of compiler comprises various phases as shown in Figure 1.7.

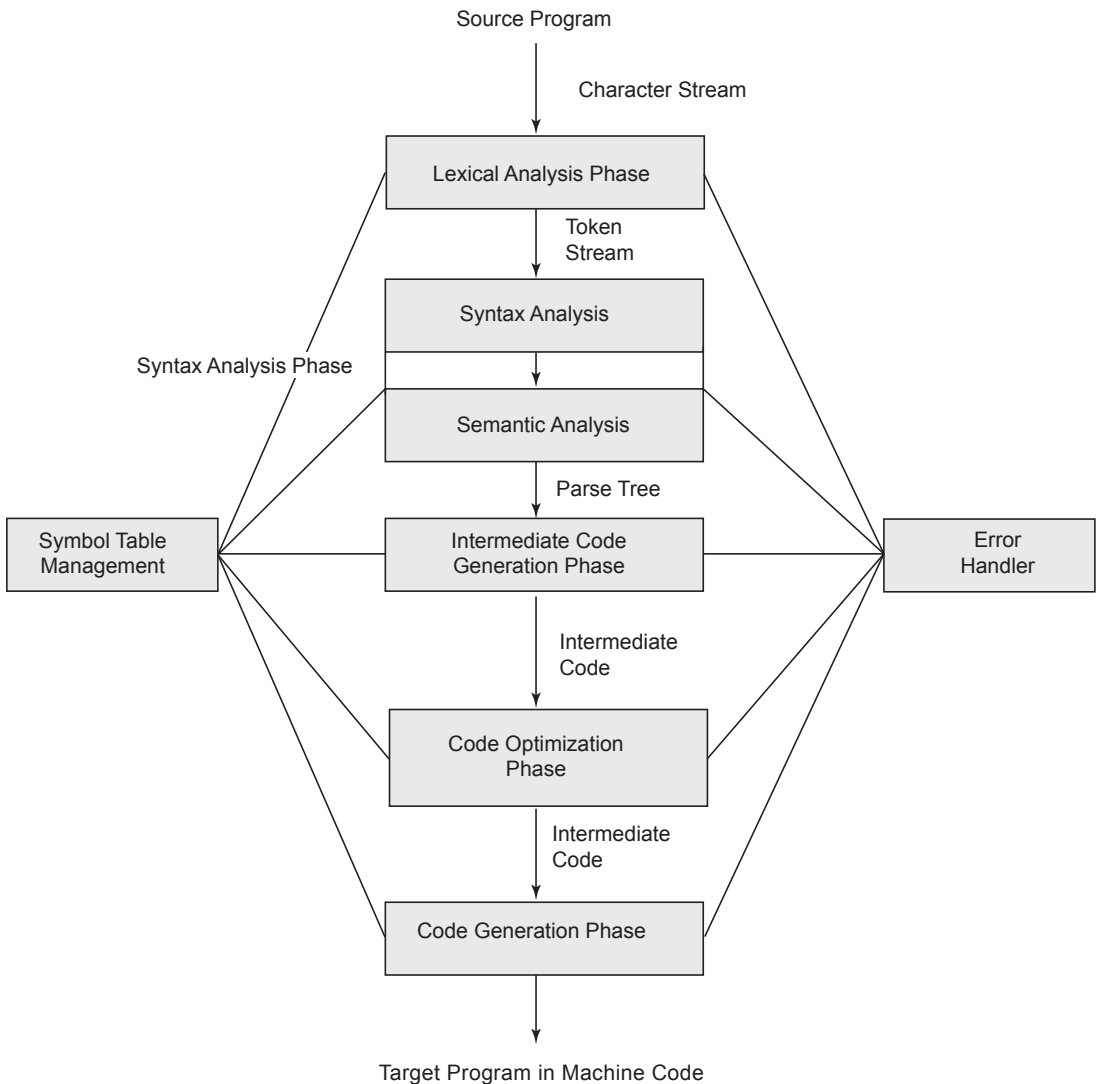


Figure 1.7 Phases of a Compiler

- ❑ **Lexical analysis phase:** Lexical analysis (also known as **scanning**) is the first phase of a compiler. Lexical analyzer or scanner reads the source program in the form of character stream and groups the logically related characters together that are known as **lexemes**. For each lexeme, a **token** is generated by the lexical analyzer. A stream of tokens is generated as the output of the lexical analysis phase, which acts as an input for the syntax analysis phase. Tokens can be of different types, namely, *keywords*, *identifiers*, *constants*, *punctuation symbols*, *operator symbols*, etc. The syntax for any token is:

```
(token_name, value)
```

where `token_name` is the name or symbol which is used during the syntax analysis phase and `value` is the location of that token in the symbol table.

- ❑ **Syntax analysis phase:** Syntax analysis phase is also known as **parsing**. Syntax analysis phase can be further divided into two parts, namely, *syntax analysis* and *semantic analysis*.
 - **Syntax analysis:** Parser uses the `token_name` token from the token stream to generate the output in the form of a tree-like structure known as **syntax tree** or **parse tree**. The parse tree illustrates the grammatical structure of the token stream.
 - **Semantic analysis:** Semantic analyzer uses the parse tree and symbol table for checking the semantic consistency of the language definition of the source program. The main function of the semantic analysis is type checking in which semantic analyzer checks whether the operator has the operands of matching type. Semantic analyzer gathers the type information and saves it either in the symbol table or in the parse tree.
- ❑ **Intermediate code generation phase:** In intermediate code generation phase, the parse tree representation of the source code is converted into low-level or machine-like intermediate representation. The intermediate code should be easy to generate and easy to translate into machine language. There are several forms for representing the intermediate code. Three address code is the most popular form for representing intermediate code. An example of three address code language is given below.

```
x1 = x2 + id
id1 = x3
```

- ❑ **Code optimization phase:** Code optimization phase, which is an optional phase, performs the optimization of the intermediate code. Optimization means making the code shorter and less complex, so that it can execute faster and takes lesser space. The output of the code generation phase is also an intermediate code, which performs the same task as the input code, but requires lesser time and space.
- ❑ **Code generation phase:** Code generation phase translates the intermediate code representation of the source program into the target language program. If the target program is in machine language, the code generator produces the target code by assigning registers or memory locations to store variables defined in the program and to hold the intermediate computation results. The machine code produced by the code generation phase can be executed directly on the machine.

Symbol table management: A symbol table is a data structure that is used by the compiler to record and collect information about source program constructs like variable names and all of its attributes, which provide information about the storage space occupied by a variable (name, type, and scope of the variables). A symbol table should be designed in an efficient way so that it permits the compiler to locate the record for each token name quickly and to allow rapid transfer of data from the records.

Error handler: Error handler is invoked whenever any fault occurs in the compilation process of source program.

Both the symbol table management and error handling mechanisms are associated with all phases of the compiler.

7. Discuss the action taken by every phase of compiler on the following instruction of source program while compilation.

Total = number1 + number2 * 5

Ans: Consider the source program as a stream of characters.

Total = number1 + number2 * 5

- ❑ **Lexical analysis phase:** Stream of characters (source program) acts as an input for the lexical analyzer, which produces the token stream as output (see Figure 1.8).
- ❑ **Syntax analysis phase:** The token stream acts as the input for the syntax analyzer. Output of the syntax analyzer is a parse tree (see Figure 1.9(a)) that acts as the input for the semantic analyzer; the output of the semantic analyzer is also a parse tree after type checking (see Figure 1.9(b)).

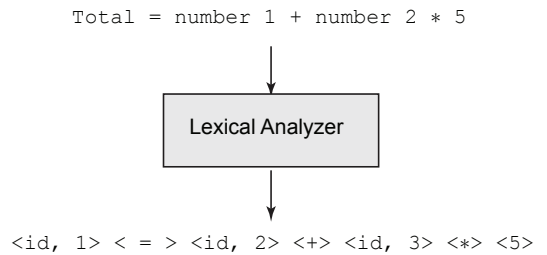


Figure 1.8 Lexical Analysis Phase

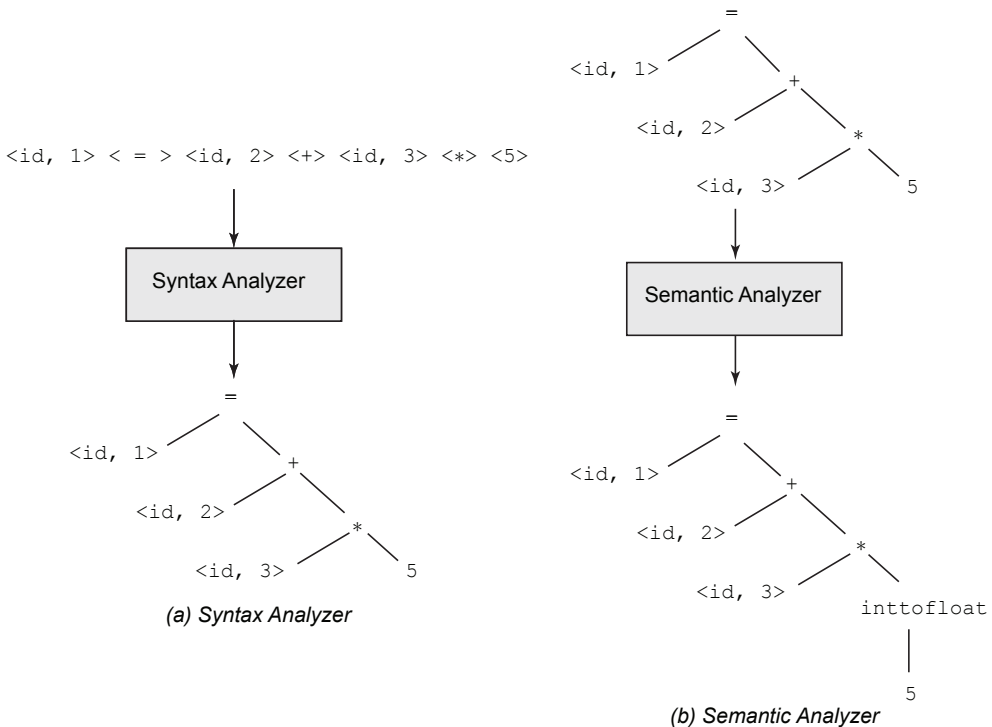


Figure 1.9 Syntax Analysis Phase

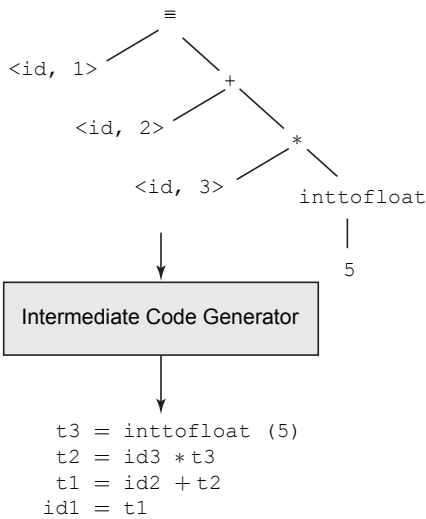


Figure 1.10 Intermediate Code Generation Phase

- ❑ **Intermediate code generation phase:** The parse tree acts as the input for the intermediate code generator, which produces an intermediate code as output (see Figure 1.10).
- ❑ **Code optimization phase:** The intermediate code of the source program acts as the input for the code optimizer. The output of the code optimizer is also an intermediate code (see Figure 1.11), that takes lesser space and lesser time to execute, and does the same task as the input intermediate code.
- ❑ **Code generation phase:** The optimized code acts as the input for the code generator. The output of the code generator is the machine language code (see Figure 1.12), known as the target program, which can be directly executed.

Note that the first operand in each instruction specifies a destination, and F in each instruction indicates that it deals with floating-point numbers.

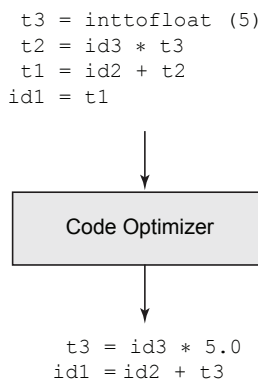


Figure 1.11 Code Optimization Phase

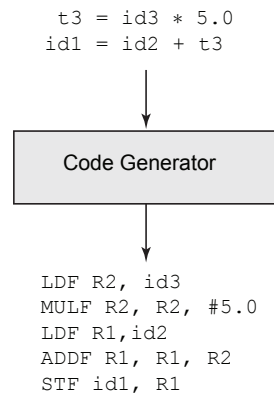


Figure 1.12 Code Generation Phase

8. What is a pass in the compilation process? Compare and contrast the features of a single-pass compiler with multi-pass compiler.

Ans: In an implementation of a compiler, the activities of one or more phases are combined into a single module known as a **pass**. A pass reads the input, either as a source program file or as the output of the previous pass, transforms the input and writes the output into an intermediate file. The intermediate file acts as either the input for the next pass or the final machine code.

When all the phases of a compiler are grouped together into a single pass, then that compiler is known as **single-pass compiler**. On the other hand, when different phases of a compiler are grouped together into two or more passes, then that compiler is known as **multi-pass compiler**.

A single-pass compiler is faster than the multi-pass compiler because in multi-pass compiler each pass reads and writes an intermediate file, which makes the compilation process time consuming. Hence, time required for compilation increases with the increase in the number of passes in a compiler.

A single-pass compiler takes more space than the multi-pass compiler because in multi-pass compiler the space used by the compiler during one pass can be reused by the subsequent pass. So, for computers having small memory, multi-pass compilers are preferred. On the other hand, for computers having large memory, single-pass compiler or compiler with fewer number of passes can be used.

In a single-pass compiler, the complicated optimizations required for high quality code generation are not possible. To count the exact number of passes for an optimizing compiler is a difficult task.

9. What are the various compiler construction tools?

Ans: For the construction of a compiler, the compiler writer uses different types of software tools that are known as **compiler construction tools**. These tools make use of specialized languages for specifying and implementing specific components, and most of them use sophisticated algorithms. The tools should hide the details of the algorithm used and produce component in such a way that they can be easily integrated into the rest of the compiler. Some of the most commonly used compiler construction tools are:

- ❑ **Scanner generators:** They automatically produce lexical analyzers or scanners.
- ❑ **Parser generators:** They produce syntax analyzers or parsers.
- ❑ **Syntax-directed translation engines:** They produce a collection of routines, which traverses the parse tree and generates the intermediate code.
- ❑ **Code generators:** They produce a code generator from a set of rules that translates the intermediate language instructions into the equivalent machine language instructions for the target machine.
- ❑ **Data-flow analysis engines:** They gather the information about how the data is transmitted from one part of the program to another. For code optimization, data-flow analysis is a key part.
- ❑ **Compiler-construction toolkits:** They provide an integrated set of routines for construction of the different phases of a compiler.

10. What is a cross compiler? Explain the concept of bootstrapping.

Ans: A compiler which may run on one machine and produce the target code for another machine is known as **cross compiler**. For example, a number of minicomputer and microprocessor compilers are implemented in such a way that they run on bigger machines and the output produced by them acts as an object code for smaller machines. Thus, the cross compilation technique facilitates platform independence. A cross compiler can be represented with the help of a T diagram as shown in Figure 1.13. It consists of three symbols S, T and I, where:

- ❑ S is the source language in which the source program is written,
- ❑ T is the target language in which the compiler produces its output or target program, and
- ❑ I is the implementation language in which compiler is written.

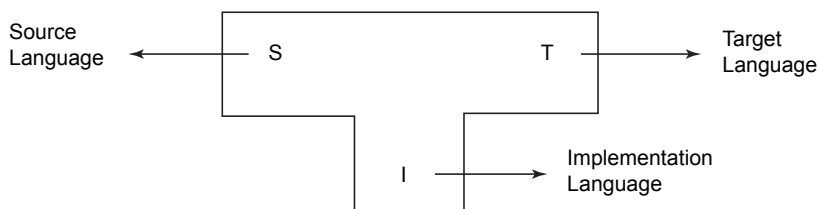


Figure 1.13 T Diagram Representation

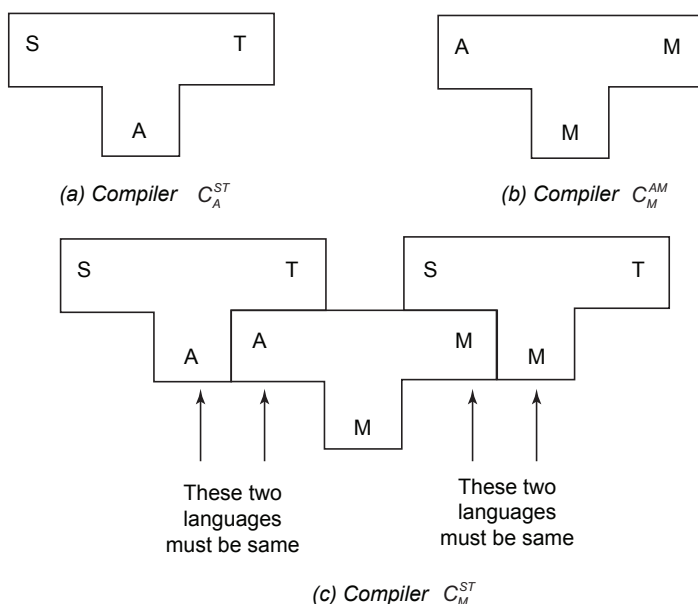


Figure 1.14 Bootstrapping

Bootstrapping: Bootstrapping is an important concept for building a new compiler. This concept uses a simple language to translate complicated programs which can further handle more complicated programs. The process of bootstrapping can be better understood with the help of an example given here.

Suppose we want to create a cross compiler for the new source language S that generates a target code in language T , and the implementation language of this compiler is A . We can represent this compiler as C_A^{ST} (see Figure 1.14(a)). Further, suppose we already have a compiler written for language A with both target and implementation language as M . This compiler can be represented as C_M^{AM} (see Figure 1.14(b)). Now, if we run C_A^{ST} with the help of C_M^{AM} , then we get a compiler C_M^{ST} (see Figure 1.14(c)). This compiler compiles a source program written in language S and generates the target code in T , which runs on machine M (that is, the implementation language for this compiler is M).

11. Explain error handling in compiler.

Ans: Error detection and reporting of errors are important functions of the compiler. Whenever an error is encountered during the compilation of the source program, an error handler is invoked. Error handler generates a suitable error reporting message regarding the error encountered. The error reporting message allows the programmer to find out the exact location of the error. Errors can be encountered at any phase of the compiler during compilation of the source program for several reasons such as:

- In lexical analysis phase, errors can occur due to misspelled tokens, unrecognized characters, etc. These errors are mostly the typing errors.
- In syntax analysis phase, errors can occur due to the syntactic violation of the language.
- In intermediate code generation phase, errors can occur due to incompatibility of operands type for an operator.

- ❑ In code optimization phase, errors can occur during the control flow analysis due to some unreachable statements.
- ❑ In code generation phase, errors can occur due to the incompatibility with the computer architecture during the generation of machine code. For example, a constant created by compiler may be too large to fit in the word of the target machine.
- ❑ In symbol table, errors can occur during the bookkeeping routine, due to the multiple declaration of an identifier with ambiguous attributes.

Multiple-Choice Questions

1. A translator that takes as input a high-level language program and translates into machine language in one step is known as _____.
 - (a) Compiler
 - (b) Interpreter
 - (c) Preprocessor
 - (d) Assembler
2. _____ create a single program from several files of relocated machine code.
 - (a) Loaders
 - (b) Assemblers
 - (c) Link editors
 - (d) Preprocessors
3. A group of logically related characters in the source program is known as _____.
 - (a) Token
 - (b) Lexeme
 - (c) Parse tree
 - (d) Buffer
4. The _____ uses the parse tree and symbol table checking the semantic consistency of the source program.
 - (a) Lexical analyzer
 - (b) Intermediate code generator
 - (c) Syntax translator
 - (d) Semantic analyzer
5. The _____ phase converts an intermediate code into an optimized code that takes lesser space and lesser time to execute.
 - (a) Code optimization
 - (b) Syntax directed translation
 - (c) Code generation
 - (d) Intermediate code generation
6. _____ is invoked whenever any fault occurs in the compilation process of source program.
 - (a) Syntax analyzer
 - (b) Code generator
 - (c) Error handler
 - (d) Lexical analyzer
7. In compiler, the activities of one or more phases are combined into a single module known as a _____.
 - (a) Phase
 - (b) Pass
 - (c) Token
 - (d) Macro
8. For the construction of a compiler, the compiler writer uses different types of software tools that are known as _____.
 - (a) Compiler writer tools
 - (c) Programming tools
 - (b) Compiler construction tools
 - (d) None of these

9. A compiler that runs on one machine and produces the target code for another machine is known as _____.
- (a) Cross compiler (b) Linker
(c) Preprocessor (d) Assembler
10. If we run a compiler C_A^{ST} with the help of another compiler C_M^{AM} , then we get a new compiler that is _____.
- (a) C_M^{SM} (b) C_A^{ST}
(c) C_M^{ST} (d) C_M^{AM}

Answers

1. (a) 2. (c) 3. (b) 4. (d) 5. (a) 6. (c) 7. (b) 8. (c) 9. (a) 10. (c)

Lexical Analysis

1. What is the role of a lexical analyzer?

Ans: The lexical analysis is the first phase of a compiler where a lexical analyzer acts as an interface between the source program and the rest of the phases of compiler. It reads the input characters of the source program, groups them into lexemes, and produces a sequence of tokens for each lexeme. The tokens are then sent to the parser for syntax analysis. If lexical analyzer is placed as a separate pass in the compiler, it would require an intermediate file to place its output, from which the parser would then take its input. To eliminate the need for the intermediate file, the lexical analyzer and the syntactic analyzer (parser) are often grouped together into the same pass where the lexical analyzer operates either under the control of the parser or as a subroutine with the parser.

The parser requests the lexical analyzer for the next token, whenever it needs one. The lexical analyzer also interacts with the symbol table while passing tokens to the parser. Whenever a token is found, the lexical analyzer returns a representation for that token to the parser. If the token is a simple construct such as parentheses, comma, or a colon, then it returns an integer code. If the token is a more complex element such as an identifier or another token with a value, the value is also passed to the parser. The lexical analyzer provides this information by calling a bookkeeping routine which installs the actual value in the symbol table if it is not already there.

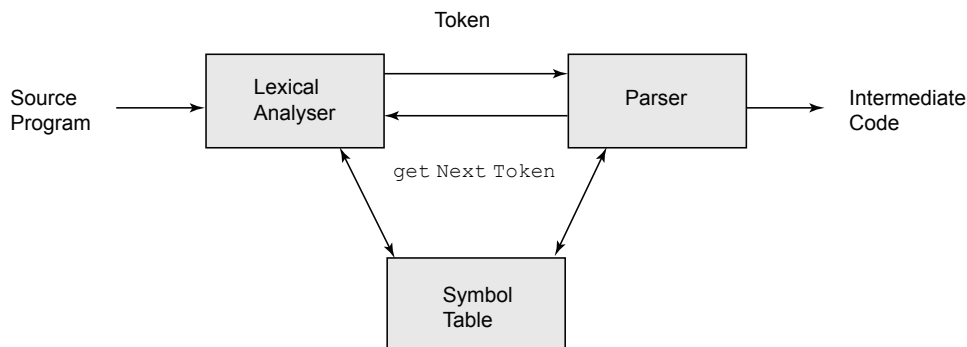


Figure 2.1 Role of the Lexical Analyzer

Besides generation of tokens, the lexical analyzer also performs certain other tasks such as:

- ❑ Stripping out comments and whitespace (tab, newline, blank, and other characters that are used to separate tokens in the input).
- ❑ Correlating error messages that are generated by the compiler during lexical analysis with the source program. For example, it can keep track of all newline characters so that it can associate an ambiguous statement line number with each error message.
- ❑ Performing the expansion of macros, in case macro preprocessors are used in the source program.

2. What do you understand by the terms tokens, patterns, and lexemes?

Ans: Tokens: The lexical analyzer separates the characters of the source language into groups that logically belong together, commonly known as **tokens**. A token consists of a token name and an optional attribute value. The **token name** is an abstract symbol that represents a kind of lexical unit and the optional attribute value is commonly referred to as **token value**. Each token represents a sequence of characters that can be treated as a single entity. Tokens can be identifiers, keywords, constants, operators, and punctuation symbols such as commas and parenthesis. In general, the tokens are broadly classified into two types:

- ❑ Specific strings such as if, else, comma, or a semicolon.
- ❑ Classes of strings such as identifiers, constants, or labels.

For example, consider an assignment statement in C

```
total = number1 + number2 * 5
```

After lexical analysis, the tokens generated are as follows:

```
<id,1> <=> <id,2> <+> <id,3> <*> <5>
```

Patterns: A rule that defines a set of input strings for which the same token is produced as output is known as **pattern**. Regular expressions play an important role for specifying patterns. If a keyword is considered as a token, the pattern is just the sequence of characters. But for identifiers and some other tokens, the pattern forms a complex structure.

Lexemes: A lexeme is a group of logically related characters in the source program that matches the pattern for a token. It is identified as an instance of that token by the lexical analyzer. For example, consider a C statement:

```
printf("Total = %d\n", total);
```

Here, `printf` is a keyword; parentheses, semicolon, and comma are punctuation symbols; `total` is a lexeme matching the pattern for token **id**; and `"Total = %d\n"` is a lexeme matching the pattern for token **literal**.

Some examples of tokens, patterns, and lexemes are given in Table 2.1.

Table 2.1 Examples of Tokens, Patterns and Lexemes

Token	Informal Description	Sample Lexeme
while	Characters w, h, i, l, e	while
then	Characters t, h, e, n	then
comparison	< or > or <= or >= or == or !=	<=, !=
id	Letter followed by letters and digits	total, number1
number	Any numeric constant	50, 3.12134, 0, 4.02e45
literal	Anything within double quotes(" ") except "	"Total"

3. What is the role of input buffering scheme in lexical analyzer?

Ans: The lexical analyzer scans the characters of source program one by one to find the tokens. Moreover, it needs to look ahead several characters beyond the next token to determine the next token itself. So, an input buffer is needed by the lexical analyzer to read its input. In a case of large source program, significant amount of time is required to process the characters during the compilation. To reduce the amount of overhead needed to process a single character from input character stream, specialized buffering techniques have been developed. An important technique that uses two input buffers that are reloaded alternately is shown in Figure 2.2.

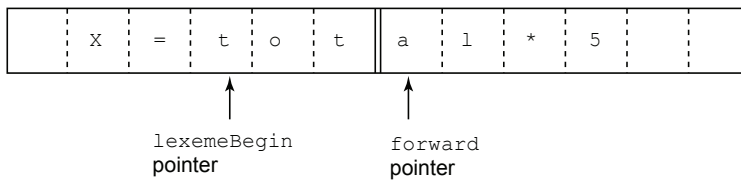


Figure 2.2 Input Buffer

Each buffer is of the same size N , where N is the size of a disk block, for example 1024 bytes. Thus, instead of one character, N characters can be read at a time. The pointers used in the input buffer for recognizing the lexeme are as follows:

- Pointer **lexemeBegin** points the beginning of the current lexeme being discovered.
- Pointer **forward** scans ahead until a pattern match is found for lexeme.

Initially, both pointers point to the first character of the next lexeme to be found. The **forward** pointer is scanned ahead until a match for a pattern is found. After the lexeme is processed, both the pointers are set to the character following that processed lexeme. For example, in Figure 2.2 the **lexemeBegin** pointer is at character **t** and **forward** pointer is at character **a**. The **forward** pointer is scanned until the lexeme **total** is found. Once it is found, both these pointers point to *****, which is the next lexeme to be discovered.

4. What are strings and languages in lexical analysis? What are the operations performed on the languages?

Ans: Before defining the terms strings and languages, it is necessary to understand the term alphabet. An **alphabet** (or **character class**) denotes any finite set of symbols. Symbols include letters, digits, punctuation, etc. The ASCII, Unicode, and EBCDIC are the most important examples of alphabet. The set $\{0, 1\}$ is the binary alphabet.

A **string** (also termed as **sentence** or **word**) is defined as a finite sequence of symbols drawn from an alphabet. The length of a string s is measured as the number of occurrences of symbols in s and is denoted by $|s|$. For example, the word 'orange' is a string of length six. The *empty string* (ϵ) is the string of length zero.

A **language** is any finite set of strings over some specific alphabet. This is an enormously broad definition. Simple sets such as ϕ , the empty set, or $\{\epsilon\}$, the set containing only the empty string, are also the languages under this definition.

In lexical analysis, there are several important operations like union, concatenation, and closure that can be applied to languages. **Union** operation means taking all the strings of both the set of languages and creating a new set of language containing all the strings. The **concatenation** of languages is done

Table 2.2 Operations on Languages

Operation	Definition	Example: Let $P = \{A, B, \dots, Z, a, b, \dots, z\}$ and $Q = \{0, 1, 2, \dots, 9\}$
Union of P and Q	$P \cup Q = \{s \mid s \text{ is in } P \text{ or } s \text{ is in } Q\}$	$P \cup Q$ is the set of letters and digits, with 62 strings of length one.
Concatenation of P and Q	$PQ = \{st \mid s \text{ is in } P \text{ and } t \text{ is in } Q\}$	PQ is the set of 520 strings of length two, consisting one letter followed by one digit.
Kleene closure of P	$P^* = \bigcup_{i=0}^{\infty} P^i$	P^* is the set of all strings of letters, including ϵ , the empty string. or $P(P \cup Q)^*$ is the set of all strings of letters and digits beginning with a letter.
Positive closure of Q	$Q^+ = \bigcup_{i=1}^{\infty} Q^i$	Q^+ is the set of all strings of one or more digits.

by concatenating a string from the first language and a string from the second language forming the new strings, in all possible ways. The (Kleene) closure of a language P , denoted by P^* , is the set of strings achieved by concatenating P zero or more times. P^0 , 'the concatenation of P zero times,' is defined to be $\{\epsilon\}$. The positive closure, denoted by P^+ , is same as the Kleene closure, but without the term P^0 , precisely, P^+ is $P^{i-1}P$. ϵ will not be in P^+ , unless it is in P itself. These operations are listed in Table 2.2.

5. Define the following terms in context of a string: prefix, suffix, substring, and subsequence.

Ans: Prefix: If zero or more symbols are removed from the end of any string s , a new string is obtained known as a prefix of string s . For example, *app*, *apple*, and ϵ are prefixes of *apple*.

Suffix: If zero or more symbols are removed from the beginning of any string s , a new string is obtained known as suffix of string s . For example, *ple*, *apple*, and ϵ are suffixes of *apple*.

Substring: If we delete any prefix and any suffix from a string s , we obtain a new string known as substring of s . For example, *pp*, *apple*, and ϵ are substrings of *apple*.

Subsequence: If we delete zero or more not necessarily consecutive positions of a string s , a new string is formed known as subsequence of s . For example, *ale* is a subsequence of *apple*.

6. What do you mean by a regular expression? Write a regular expression over alphabet $\Sigma = (x, y, z)$ that represents all strings of length three.

Ans: A **regular expression** is a compact notation that is used to represent the patterns corresponding to a token. It is used to describe all the languages that can be built by applying union, concatenation, and closure operations to the symbols of some alphabet. The regular expression represents pattern to define the language which includes a set of strings. The strings are considered to be in the said language if they match the pattern; otherwise, they are not in the said language.

For example, consider the identifiers in a programming language, where an identifier may consist of a letter or more followed by any number of digits or an underscore (`_`). Thus, the language for C identifiers can be described as:

`letter_(letter_|digit)*`

Here, the vertical bar indicates union and the star indicates zero or more instances. The parentheses are used to group subexpressions.

There exist some primitive regular expressions which are of universal type, over some alphabet Σ , which are defined as follows:

- x (for each $x \in \Sigma$), the primitive regular expression x defines the language $\{x\}$, that is, the only string is ' x ' in this particular language which is of length one.
- λ (empty string), the primitive regular expression λ defines the language $\{\lambda\}$, that is, the only string is the empty string in this particular language. The language denoted by λ is of universal type.
- ϕ (indicates no string at all), the primitive regular expression ϕ denotes the language $\{\}$, that is, no string at all in this particular language. The language denoted by ϕ is also of universal type.

Thus, it must be noted that if $|\Sigma| = \text{number of symbols present in it} = n$, then there are $n + 2$ primitive regular expressions defined over it.

Construction of Regular Expression

Given $\Sigma = \{x, y, z\}$.

And, we have to construct a regular expression that represents all strings of length three.

For this, let us choose three arbitrary symbols a_1, a_2, a_3 .

Thus, the regular expression will be:

$$a_1 a_2 a_3,$$

where

$$\begin{aligned} a_1 &= \text{either 'x' or 'y' or 'z'} \\ a_2 &= \text{either 'x' or 'y' or 'z'} \\ a_3 &= \text{either 'x' or 'y' or 'z'} \end{aligned}$$

7. List the rules for constructing regular expressions. Write some properties to compose additional regular expressions. What is a regular definition? Give a suitable example.

Ans: The rules for constructing regular expressions over some alphabet Σ are divided into two major classifications which are as follows:

- (i) Basic rules (ii) Induction rules

Basic rules: There are two rules that form the basis:

1. ϵ is a regular expression, and $L(\epsilon)$ is $\{\epsilon\}$, that is, its language contains only an empty string.
2. If a is a symbol in Σ , then a is a regular expression, and $L(a) = \{a\}$, which implies the language with one string, of length one, with a in its one position.

Induction rules: There are four induction rules that built larger regular expressions recursively from smaller regular expressions. Suppose R and S are regular expressions with languages $L(R)$ and $L(S)$, respectively.

1. $(R)(S)$ is a regular expression representing the language $L(R) \cdot L(S)$.
2. $(R) | (S)$ is a regular expression representing the language $L(R) \cup L(S)$.
3. $(R)^*$ is a regular expression representing the language $(L(R))^*$.
4. (R) is a regular expression representing $L(R)$. This rule states that additional pairs of parentheses can be added around expressions without modifying the language.

Properties of Regular Expression: To compose additional regular expressions, the following properties are to be considered, a finite number of times:

1. If a_1 is a regular expression, then (a_1) is also a regular expression.
2. If a_1 is a regular expression, then a_1^* is also a regular expression.
3. If a_1 and a_2 are two regular expressions, then a_1a_2 is also a regular expression.
4. If a_1 and a_2 are two regular expressions, then $a_1 + a_2$ is also a regular expression.

Regular Definition: If Σ = alphabet set, then a regular definition is a sequence of definitions of the form:

$$\begin{aligned} D_1 &\rightarrow R_1 \\ D_2 &\rightarrow R_2 \\ &\dots \\ D_n &\rightarrow R_n \end{aligned}$$

where

- D_i is a new symbol, not in Σ and not the same as any of the other D 's.
- R_i is a regular expression over the alphabet $\Sigma \cup \{D_1, D_2, \dots, D_{i-1}\}$.

For example, let us consider the C identifiers that are strings of letters, digits, and underscores. Here, we give a regular definition for the language of C identifiers.

$$\begin{aligned} \text{letter_} &\rightarrow A \mid B \mid \dots \mid Z \mid a \mid b \mid \dots \mid z \mid _ \\ \text{digit} &\rightarrow 0 \mid 1 \mid \dots \mid 9 \\ \text{id} &\rightarrow \text{letter_}(\text{letter_}|\text{digit})^* \end{aligned}$$

8. What is a transition diagram? Draw a transition diagram to identify the keywords IF, THEN, ELSE, DO, WHILE, BEGIN, END.

Ans: While constructing a lexical analyzer, we represent patterns in the form of flowcharts, called transition diagrams. A **transition diagram** consists of a set of nodes and edges that connect one state to another. A node (or a circle) in a transition diagram represents a state and each edge (or an arrow) represents the transition from one state to another. Each edge is labeled with one or more symbols.

A **state** is basically a condition that could occur while scanning the input to find out a lexeme that matches one of the several patterns. We can also think of a state as summarizing all we need to know what characters have been seen between the `lexemeBegin` pointer and the `forward` pointer. Suppose, currently we are at state q_i , and the next input symbol is a , then we look for an edge e coming out of the current state q_i that is having the label a . If such an edge is found, then we move ahead the `forward` pointer and enter the state of the transition diagram to which this edge is connected.

Among all the states, one state, say q_0 , is termed as initial or start state. The transition diagram always begins in the start state before any input symbols have been read. One or more states are said to be final or accepting states and are represented by double circles. We may also attach actions to the final states to indicate that a token and an attribute value are being returned to the parser. In some cases, it is also necessary to move the `forward` pointer backward by certain number of positions, then we can place that many number of $*$'s near the final state. For example, if we want to retract the pointer by one position, then we can place a single $*$, for two positions, $**$ can be placed, and so on.

The transition diagram to identify the keywords BEGIN, END, IF, THEN, ELSE, DO, and WHILE is shown in Figure 2.3.

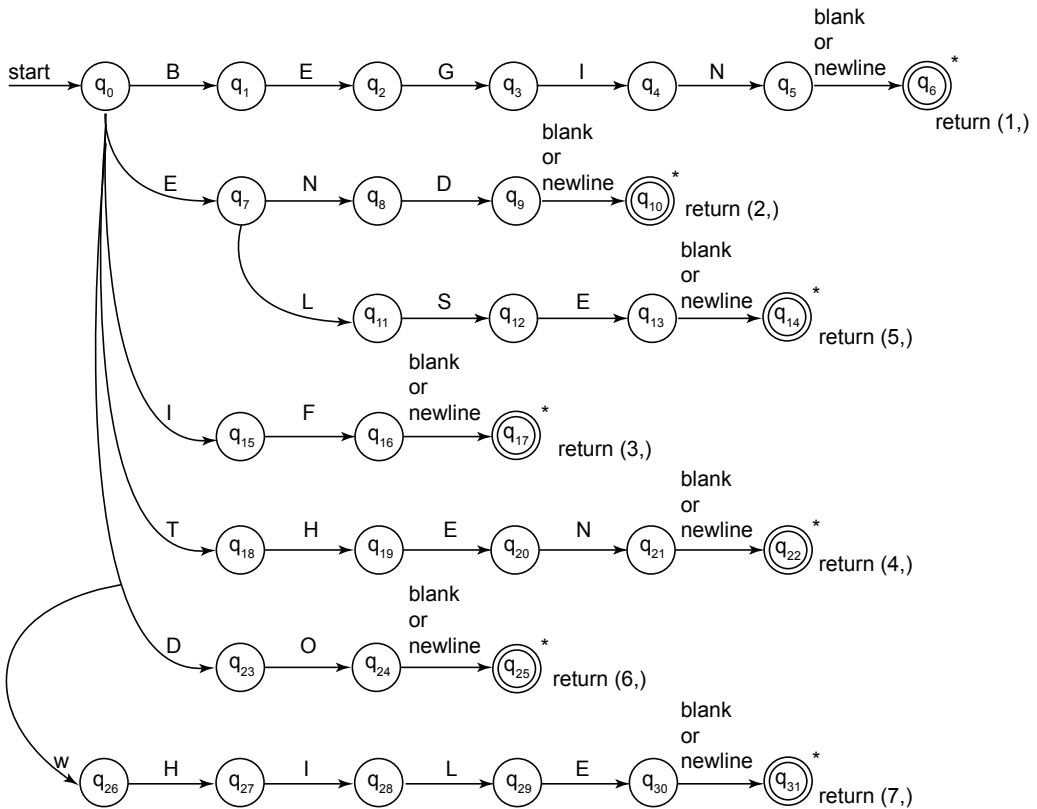


Figure 2.3 Transition Diagram to Identify Keywords

9. Draw the transition diagram for identifiers, constants, and relational operators (relops).

Ans: Transition diagram for identifiers is shown in Figure 2.4.

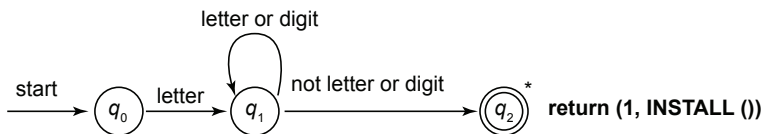


Figure 2.4 Transition Diagram for Identifiers

The transition diagram for constants is shown in Figure 2.5.



Figure 2.5 Transition Diagram for Constants

The transition diagram for relational operators (relops) is shown in Figure 2.6.

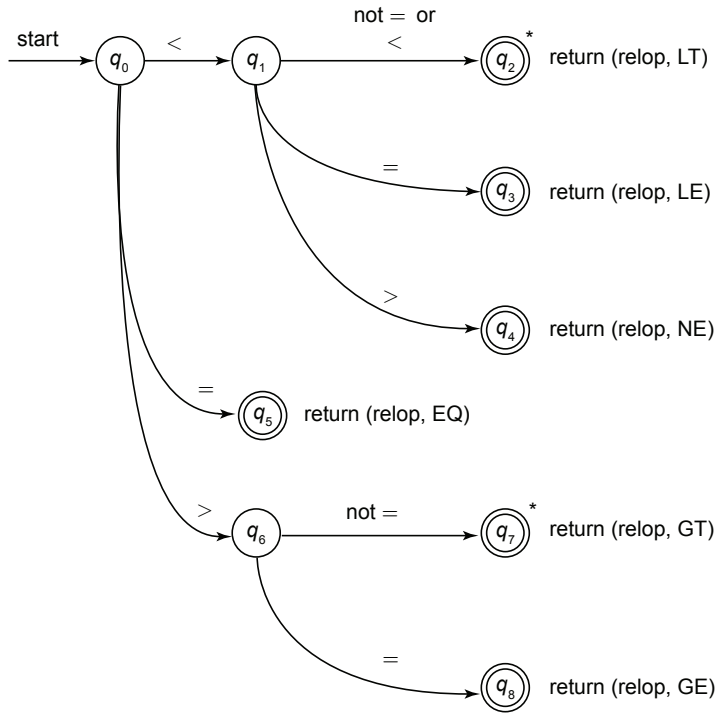


Figure 2.6 Transition Diagram for Relops

10. Draw a transition diagram for unsigned numbers.

Ans: The transition diagram for unsigned numbers is shown in Figure 2.7.

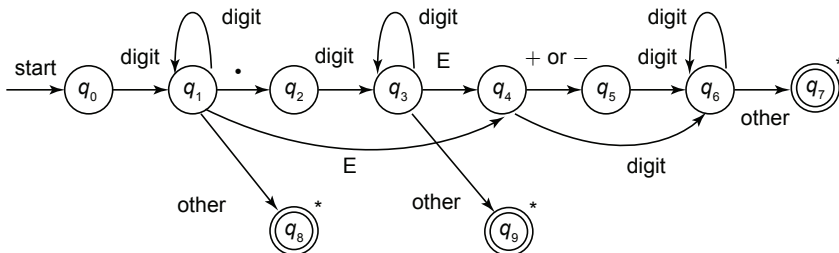


Figure 2.7 Transition Diagram for Unsigned Numbers

- ❑ In the transition diagram for unsigned numbers, we begin with the start state q_0 , if we see a digit, we move to state q_1 . In that state, we can read any number of additional digits.
- ❑ In case we see anything except a digit, dot, or E from state q_1 , it implies that we have seen an integer number, for example 789. In such case, we enter the state q_8 , where we return token number and a pointer to a table of constants where lexeme is entered.

- If we see a dot from state q_1 , then we have an ‘optional fraction,’ and we enter the state q_2 . Now if we look for one or more additional digits, we move to the state q_3 for this purpose.
- In case we see an E in state q_3 , then we have an ‘optional exponent,’ which is recognized by the states q_4 through q_7 , and return the lexeme at final state q_7 .
- In state q_3 , if we have come to an end of the fraction, and we have not seen any exponent E, we move to the state q_9 , and return the lexeme found.

11. What is a finite automata? Explain its two different types.

Ans: A **finite automata** is a recognizer for a language P that takes a string x as an input and returns ‘yes’ if x is a sentence of P , else returns ‘no’. It is a part of lexical analyzer that identifies the presence of a token on the input for the language defining that token. A regular expression can be converted to a recognizer by constructing a generalized transition diagram (that represents finite automata) from the expression. Finite automata can be described in two types, namely non-deterministic finite automata (NFA) and deterministic finite automata (DFA).

- **Non-deterministic finite automata (NFA):** A finite automata is said to be non-deterministic, if we have more than one possible transition on the same input symbol from some state. Non-deterministic finite automata (NFA) have no restrictions on the labels of their edges in a sense that the same symbol can label several edges out of the same state, and ϵ , the empty string, is also a possible label. A non-deterministic finite automata (NFA) is a set of five tuples that are represented as:

$$M = (Q, \Sigma, \delta, q_0, F)$$

where

Q is a non-empty finite set of states.

Σ is a non-empty finite set of input symbols. We assume that ϵ never belongs to Σ .

q_0 is a starting state, one of the states in Q .

F is a subset of Q containing final (or accepting) states.

δ is a transition function, which takes two arguments, a state and an input symbol from $\Sigma \cup \{\epsilon\}$, and returns a set of next states. δ is represented as:

$$\delta : Q * (\Sigma \cup \{\epsilon\}) \rightarrow 2^Q$$

Graphically, the transition function can be represented as follows:

$$\delta(q, a) \rightarrow q_0, q_1, q_2, \dots, q_n$$

- **Deterministic finite automata (DFA):** A finite automata is said to be deterministic, if corresponding to an input symbol, there is only one resultant state, thus, having only one transition. For each state, and for each symbol of its input alphabet, deterministic finite automata (DFA) can have exactly one edge with that symbol leaving that state. It is also a set of five tuples and represented as

$$M = (Q, \Sigma, \delta, q_0, F)$$

where

Q is a non-empty finite set of states.

Σ is a non-empty finite set of input symbols.

q_0 is an initial state of DFA and member of Q .

F is a subset of Q containing final states.

δ is transition function, which takes two arguments, a state and an input symbol, and returns a single state 'represented by $Q \times \Sigma \rightarrow Q$ '. Let q is the state and a be the input symbol passed to the transition function, then $\delta(q, a) = q'$, where q' is the output function, which may be same as q .

Graphically, the transition function can be represented as follows:

$$\delta(q, a) \rightarrow q'$$

DFA is a special case of an NFA where

- There are no moves on input ϵ and
- For each state q and input symbol a , there is exactly one edge out of q labeled a .

12. What do you mean by NFA with ϵ -transition?

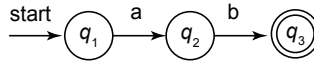
Ans: NFA with ϵ -transition is defined as a modified finite automata that permits transition without input symbols, along with zero, one or more transitions on input symbols. Let us take an example, where we have to design an NFA with ϵ -transition for the following accepting language:

$$L = \{ab \cup aab^*\}$$

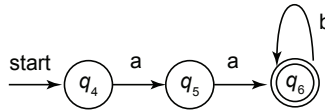
To solve this problem, first we divide the language as follows:

$$L = L_1 \cup L_2, \text{ where } L_1 = ab \text{ and } L_2 = aab^*$$

Now, we construct NFA for L_1 .



Now, we construct NFA for L_2 .



Finally, we combine the transition diagram of L_1 and L_2 , to construct the NFA with ϵ -transition for given input language as shown in Figure 2.8. In this NFA, we use ϵ -transitions to reach at states q_1 and q_4 .

13. Explain the working of ϵ -closure, with a suitable example.

Ans: NFA with ϵ -transition accepts a string w in Σ^* , if there exists at least one path which corresponds to w initializing from start state and ending at final state. If the path contains ϵ -moves, then we define a function ϵ -closure(q), where q is the state of automata. The ϵ -closure function is defined as follows:

ϵ -closure(q) = set of all those states of automata which can be reached from q on a path labeled by ϵ , that is without consuming any input symbol. For example, consider the following NFA:

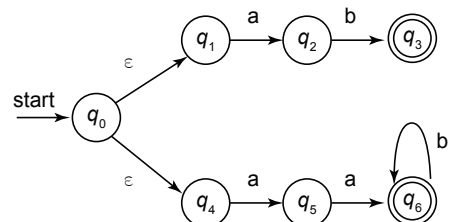
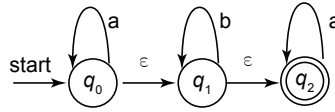


Figure 2.8 NFA with ϵ -Transition



In this NFA,

$$\epsilon\text{-closure}(q_0) = \{q_0, q_1, q_2\}$$

$$\epsilon\text{-closure}(q_1) = \{q_1, q_2\}$$

$$\epsilon\text{-closure}(q_2) = \{q_2\}$$

14. Write an algorithm to convert a given NFA into an equivalent DFA.

Or

Give the algorithm for subset construction and computation of ϵ -closure.

Ans: The basic idea behind constructing a DFA from NFA is to merge two or more states of DFA into one. To convert a given NFA into an equivalent DFA, we note that a set of states in an NFA corresponds to a state in the DFA. All the NFA states are reachable from at least one state of the same set using ϵ -transition only, without considering any further input. Moreover, from this set of states which are based on some input symbol we can reach another set of states. In the DFA, we take these sets as unique states. We define two sets that are as follows:

- $\epsilon\text{-closure}(q)$: In an NFA, ϵ -closure of a state q defined to be the set of states (including q) that are reachable from q using ϵ -transitions only.
- $\epsilon\text{-closure}(Q)$: ϵ -closure of a set of states Q of an NFA is defined to be the set of states reachable from any state in Q using ϵ -transitions only.

The algorithm for computing ϵ -closure of a set of states Q is given in Figure 2.9.

```

 $\epsilon\text{-closure}(Q) = Q$ 
Set all the states of  $\epsilon\text{-closure}(Q)$  unmarked
For each unmarked state  $q$  in  $\epsilon\text{-closure}(Q)$  do
  Begin
    Mark  $q$ 
    For each state  $q'$  having an edge from  $q$  to  $q'$  labeled  $\epsilon$  do
      Begin
        If  $q'$  is not in  $\epsilon\text{-closure}(Q)$  then
          Begin
            add  $q'$  to  $\epsilon\text{-closure}(Q)$ 
            Set  $q'$  unmarked
          End
        End
      End
    End
  End

```

Figure 2.9 Algorithm for Computing ϵ -Closure

Now, to convert an NFA to the corresponding DFA, we consider the algorithm shown in Figure 2.10.

Input: An NFA with set of states Q , start state q_0 , set of final states F

Output: Corresponding DFA with start state d_0 , set of states Q_D , set of final states F_D

```

Begin
   $d_0 = \epsilon\text{-closure}(q_0)$ 
   $Q_D = \{d_0\}$ 
  If  $d_0$  contains a state from  $F$  then  $F_D = \{d_0\}$  else  $F_D = \phi$ 
  Set  $d_0$  unmarked
  While there are unmarked states in  $Q_D$  do
    Begin
      Let  $d$  be such a state
      For each input symbol  $x$  do
        Begin
          Let  $S$  be the set of states in  $Q$  having transitions on  $x$  from
          any state of the NFA corresponding to the DFA state  $d$ 
           $d' = \epsilon\text{-closure}(S)$ 
          If  $d'$  is already present in  $Q_D$  then
            add the transition  $d \rightarrow d'$  labeled  $x$ 
          else
            Begin
               $Q_D = Q_D \cup \{d'\}$ 
              add the transition  $d \rightarrow d'$  labeled  $x$ 
              Set  $d'$  unmarked
              If  $d'$  contains a state of  $F$  then  $F_D = F_D \cup \{d'\}$ 
            End
          End
        End
      End
    End
  End

```

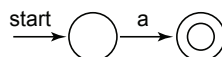
Figure 2.10 Algorithm to Convert NFA to DFA

15. Give Thompson's construction algorithm. Explain the process of constructing an NFA from a regular expression.

Ans: To construct an NFA from a regular expression, we present a technique that can be used as a recognizer for the tokens corresponding to a regular expression. In this technique, a regular expression is first broken into simpler subexpressions, then the corresponding NFA are constructed and finally, these small NFAs are combined with the help of regular expression operations. This construction is known as **Thompson's construction**.

Thompson's construction algorithm: The brief description of Thompson's construction algorithm is as follows:

- Step 1:** Find the alphabet set Σ from the given regular expression. For example, for the regular expression $a (a \mid b)^* ab$, $\Sigma = \{a, b\}$. Now, determine all primitive regular expressions.
- Step 2:** Construct equivalent NFAs for all primitive regular expressions. For example, an equivalent NFA for the primitive regular expression 'a' is shown below:



- Step 3:** Apply the rules for union, concatenation, grouping, and (Kleene)* to get the equivalent NFA of the given regular expression.

While constructing an NFA from a regular expression using Thompson's construction, these rules are followed:

- For ϵ or any alphabet symbol x in the alphabet set Σ , the NFA consists of two states—a start state and a final state. The transition is labeled by ϵ or x as shown below:



- If we are given NFAs of two regular expressions r_1 and r_2 as $N(r_1)$ and $N(r_2)$, then we can construct a composite NFA for the regular expression $(r_1 | r_2)$ as follows:

- Add new initial state (q_0) and final state q_f .
- Introduce ϵ -transitions from q_0 to the start state of $N(r_1)$ and $N(r_2)$. Similarly, introduce ϵ -transitions from final states of $N(r_1)$ and $N(r_2)$ to the new final state q_f (see Figure 2.11).

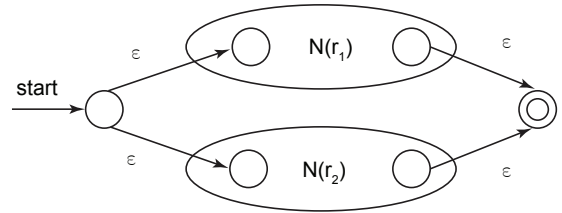


Figure 2.11 NFA for $r_1 | r_2$

Note that the final states of $N(r_1)$ and $N(r_2)$ are no longer be the final states in the composite NFA $N(r_1 | r_2)$.

- The NFA $N(r_1 r_2)$ for the regular expression $r_1 r_2$ can be constructed by merging the final state of $N(r_1)$ with the start state of $N(r_2)$. The start state of $N(r_1)$ will become the start state of new NFA and the final state of $N(r_2)$ will become the final state of new NFA as shown in Figure 2.12.

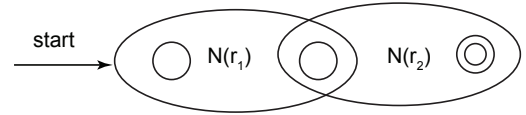


Figure 2.12 NFA for $r_1 r_2$

- If we are given NFA $N(r^*)$, we construct a regular expression r^* from the NFA $N(r)$ of r as follows:

- Add new start state (q_0) and final state (q_f).
- Introduce ϵ -transitions from q_0 to the start state of $N(r)$, from the final state of $N(r)$ to q_f , from the final state of $N(r)$ back to the start state of $N(r)$ that corresponds to repeated occurrence of r , and from q_0 to q_f corresponding to the zero-occurrence of r (see Figure 2.13).

- If $N(r)$ be the NFA for a regular expression r , it is also the NFA for the parenthesized expression (r) .

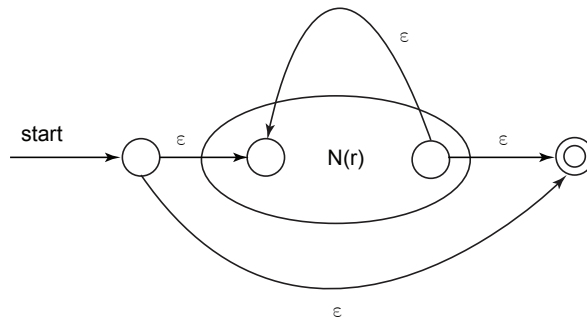


Figure 2.13 NFA for r^*

16. Explain the functions `nullable(n)`, `firstpos(n)`, `lastpos(n)`, and `followpos(p)` and describe the rules to compute them.

Ans: To convert a regular expression into DFA, we construct syntax tree of the regular expression and then compute the four functions as follows:

nullable(n): This function is true for a syntax-tree node n if and only if its subexpression contains ϵ in its language. In other words, the subexpression can be *made null* or the empty string, even if it can represent other strings as well. The rules to compute `nullable(n)` for any node n are given as follows:

- For a leaf labeled ϵ , `nullable` is true.
- For a leaf with position i , `nullable` is false because they correspond to non- ϵ operands.
- For an or-node $n = c_1 | c_2$, `nullable` will be true only if either of its child is nullable.
- For a cat-node $n = c_1 c_2$, `nullable` will be true only if both the children are nullable.
- For a star-node $n = c_1^*$, `nullable` is always true.

firstpos(n): It is the set of positions in the subtree rooted at n corresponding to the first symbol of at least one string in the language of the subexpression rooted at n . The rules to compute `firstpos(n)` for any node n are as follows:

- For a leaf labeled ϵ , `firstpos(n)` will be ϕ .
- For a leaf with position i , `firstpos(n)` will be i itself.
- For an or-node $n = c_1 | c_2$, we take the union of the `firstpos` of left child and right child.
- For a cat-node $n = c_1 c_2$, if the left child c_1 is nullable, then we take the union of `firstpos` of the left child c_1 as well as the right child c_2 , otherwise only `firstpos` of the left child c_1 is possible.
- For star-node $n = c_1^*$, we take the value of `firstpos` of the left child c_1 .

lastpos(n): It is the set of positions in the subtree rooted at n corresponding to the last symbol of at least one string in the language of the subexpression rooted at n . The rules to compute `lastpos` are the same as that of `firstpos`, except the rule for the cat-node, where the roles of its children are interchanged. That is, for a cat-node $n = c_1 c_2$, we consider whether the right child c_2 is nullable. If yes, then we take the union of `lastpos`(c_1) and `lastpos`(c_2), otherwise only `lastpos`(c_2) is possible.

followpos(p): It is set of positions q , for a position p , in the syntax tree such that there exist some string $s = x_1 x_2 \dots x_n$ in $L((r) \#)$ such that for some i , there is a way to explain the membership of s in $L((r) \#)$ by matching x_i to position p of the syntax tree and x_{i+1} to position q . To compute `followpos`, there are only two ways given as follows:

- If $n = c_1 c_2$, then for every position i in `lastpos`(c_1), `followpos`(i) will be all positions in `firstpos`(c_2).
- If n is a star-node, and i is a position in `lastpos`(n), then `followpos`(i) will be all positions in `firstpos`(n).

To understand how to compute these functions, consider the syntax tree for the expression $(x|y)^*xy\#$ shown in Figure 2.14. The numeric value associated with each leaf node indicates the position of the leaf and also the position of its symbol.

In this syntax tree, only the star-node is nullable because every star-node is nullable. All the leaf nodes correspond to non- ϵ operands; thus, none of them is nullable. The or-node is also not nullable because

neither of its child nodes is nullable. Finally, the cat-nodes also have non-nullable child nodes, and hence none of them is nullable. The *firstpos* and *lastpos* of all the nodes are shown in Figure 2.15.

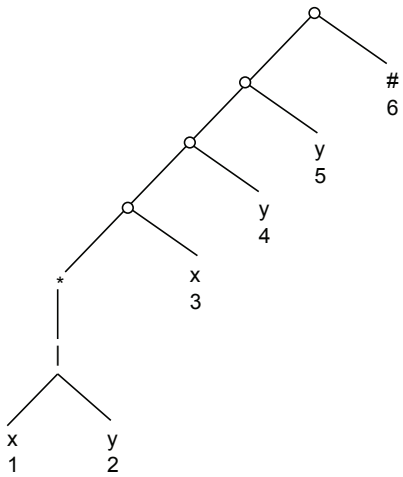


Figure 2.14 Syntax Tree for $(x|y)^*xy\#$

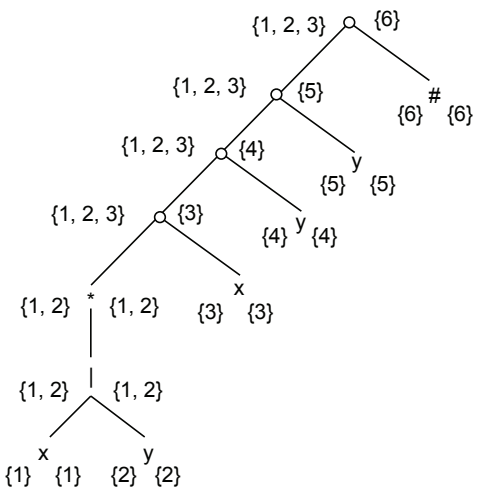


Figure 2.15 Firstpos and Lastpos for the Nodes

The *followpos* of all the leaf nodes is given in Table 2.3.

Table 2.3 followpos for the Nodes

Value of n	$\text{followpos}(n)$
1	{1,2,3}
2	{1,2,3}
3	{4}
4	{5}
5	{6}
6	ϕ

17. Describe the process of constructing a DFA directly from a regular expression.

Ans: The process for constructing a DFA directly from a regular expression consists of the following steps:

- From the augmented regular expression $(r)\#$, construct a syntax tree T rooted at node n_0 .
- For syntax tree T , compute nullable, *firstpos*, *lastpos*, and *followpos*.
- Construct D_{states} , the set of states of DFA D , and D_{tran} , the transition function for D , by using the algorithm given in Figure 2.16.

The states of D are sets of position in T . Initially, all the states are unmarked, and a state becomes marked just before its out-transitions. $\text{firstpos}(n_0)$ is set as the start state of D , and the states containing the position for the endmarker symbol $\#$ are considered as the accepting states.

```

Initialize Dstates with only the unmarked state firstpos( $n_0$ )
For each unmarked state S in Dstates do
  Begin
    Mark S
    For each input symbol x do
      Begin
        Let U be the union of followpos(p) for all p in S that cor
        respond to x
        if (U is not in Dstates) then
          add U as an unmarked state to Dstates
          Dtran[S,x] = U
      End
    End
  End

```

Figure 2.16 Algorithm for Constructing Dstates and Dtran

18. Explain lexical analyzer generator (LEX) and its structure.

Or

What is a lex compiler? Write its specification.

Ans: A **lex compiler** or simply **lex** is a tool for automatically generating a lexical analyzer for a language. It is an integrated utility of the UNIX operating system. The input notation for the lex is referred to as the **lex language**. The process of constructing a lex analyzer with the lex compiler is shown in Figure 2.17.

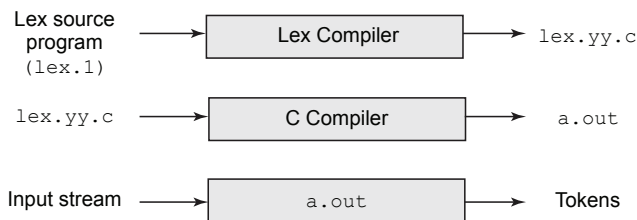


Figure 2.17 Constructing a Lexical Analyzer with Lex

The lex source program, `lex.1`, is passed through lex compiler to produce the C program file `lex.yy.c`. The file `lex.1` basically contains a set of regular expressions along with the routines for each regular expression. The routines contain a set of instructions that need to be executed whenever a token specified in the regular expression is recognized. The file `lex.yy.c` is then compiled using a C compiler to produce the lexical analyzer `a.out`. This lexical analyzer can now take a stream of input characters and produce a stream of tokens.

The lexical analyzer `a.out` is basically a function that is used as a subroutine of the parser. It returns an integer code for one of the possible token names. The attribute value for the token is stored in a global variable `yylval`. This variable is shared by both lexical analyzer and parser. This enables to return both the name and the attribute value of a token.

Lex Specifications or Structure

A lex program consists of the following form:

```

declarations
%%
translation rules
%%
auxiliary routines

```

The *declarations* section includes variable declarations, constant declarations, and regular definitions. The regular definitions can be used as parts in the *translation rules* section. This section contains the patterns and the associated action. The translation rules each have the following form:

Pattern {Action}

Here, each *pattern* is a regular expression, which may use the regular definitions of the declaration section. Each *action* specifies a set of statements to be executed whenever rule r_i matches the current input sequence.

The third section (*auxiliary routines*) holds the additional functions that may be used to write the action part. These functions can also be compiled separately and loaded with the lexical analyzer.

19. What are the proper recovery actions in lexical analysis?

Ans: The possible error recovery actions in lexical analysis phase are as follows:

- ☐ Deleting an extra character.
- ☐ Inserting a missing character.
- ☐ Replacing an incorrect character by a correct character.
- ☐ Transposing two adjacent characters.

20. Find the tokens for the given code:

```
For I = 1 to 100 do
```

Ans: The given code is shown below:

For	I	=	1	to	100	do
-----	---	---	---	----	-----	----

Tokens are:

Keywords → For, to, do

Identifiers → I

Constants → 1, 100

Operators → =

21. Construct a symbol table and find the tokens for the given code:

```
IF (i = 20) Then GOTO 100
```

Ans: The given code is shown below:

If	(	i	=	20	)	Then	GOTO	100
----	---	---	---	----	---	------	------	-----

Tokens are:

Keywords → If, Then, GOTO

Identifiers → i

Constants → 20, 100

Operators → (, =,)

The symbol table corresponding to the given code is as follows:

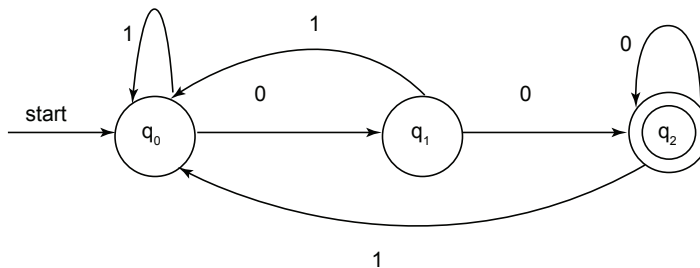
	.
	.
231	constant, integer, value = 20
	.
	.
642	label, value = 100
	.
	.
782	identifier, integer, value = i

After finding the required tokens and storing them into the symbol table, code is rewritten as follows:

```
If([identifier, 782] = [constant, 231]) Then GOTO [label, 642]
```

22. Design a Finite Automata that accepts set of strings such that every string ends with 00, over alphabets {0,1}.

Ans: Here, we have to construct a finite automata that will accept all strings like {00, 01100, 110100, . . .}. The finite automata for the given problem is given below:



Hence, finite automata $M = \{Q, \Sigma, \delta, q_0, F\}$ will be

$$Q = \{q_0, q_1, q_2\}$$

$$\Sigma = \{0, 1\}$$

$$q_0 = \{q_0\}$$

$$F = \{q_2\}$$

The transition function δ is shown with the help of the following transition table:

δ	0	1
$\rightarrow q_0$	q_1	q_0
q_1	q_2	q_0
$*q_2$	q_2	q_0

The symbol $\rightarrow$ in the table indicates that q_0 is the start state, and $*$ indicates that q_2 is the final state.

23. Design a finite automata which will accept the language

$$L = \{w \in (0, 1)^* / \text{second symbol of } w \text{ is '0' and fourth input is '1'}\}.$$

Ans: Here, we have to construct finite automata that will accept all the strings of which second symbol is 0 and fourth is 1. The finite automata for the given problem is shown below:

Hence, finite automata $M = \{Q, \Sigma, \delta, q_0, F\}$ will be

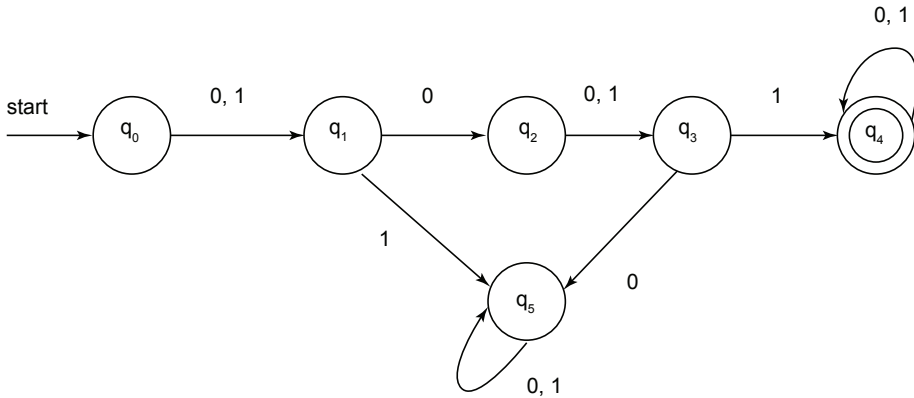
$$Q = \{q_0, q_1, q_2, q_3, q_4, q_5\}$$

$$\Sigma = \{0, 1\}$$

$$q_0 = \{q_0\}$$

$$F = \{q_4\}$$

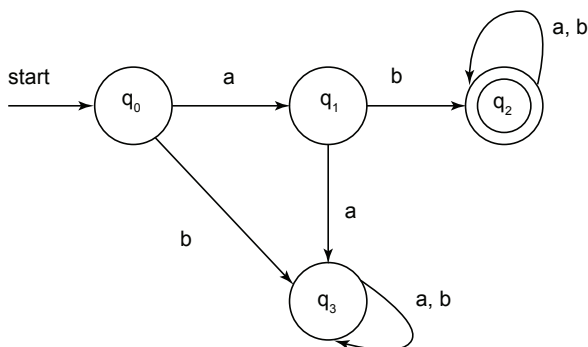
The transition function δ is shown with the help of the following transition table:



δ	0	1
$\rightarrow q_0$	q_1	q_1
q_1	q_2	q_5
q_2	q_3	q_3
q_3	q_5	q_4
$*q_4$	q_4	q_4
q_5	q_5	q_5

24. Construct a DFA for language over alphabet $\Sigma = \{a, b\}$ that will accept all strings beginning with 'ab'.

Ans: Here, we have to construct a DFA that will accept all strings beginning with ab like {ab, abb, abaab, ababb, abba, ...}.



Hence, DFA $M = \{Q, \Sigma, \delta, q_0, F\}$ will be

$$Q = \{q_0, q_1, q_2, q_3\}$$

$$\Sigma = \{a, b\}$$

$$q_0 = \{q_0\}$$

$$F = \{q_2\}$$

The transition function δ is shown with the help of the following transition table:

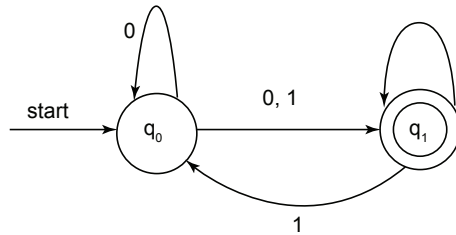
δ	a	b
$\rightarrow q_0$	q_1	q_3
q_1	q_3	q_2
$*q_2$	q_2	q_2
q_3	q_3	q_3

25. Convert the following NFA into DFA.

$M = \{\{q_0, q_1\}, \{0, 1\}, \delta, q_0\{q_1\}\}$ and δ is

States	Inputs	
	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_1\}$
q_1	ϕ	$\{q_0, q_1\}$

Ans: We will first draw the NFA according to the given transition table, as shown below:



Now, we convert the NFA into DFA by following the given steps:

Step 1: Find all the transitions from initial state q_0 for every input symbol, that is, $\Sigma = \{0, 1\}$. If we get a set having more than one state for a particular input, then we consider that set as new single state. From the given transition table, it is clear that

$$\delta(q_0, 0) \rightarrow \{q_0, q_1\}, \text{ that is, } q_0 \text{ transits to both } q_0 \text{ and } q_1 \text{ for input } 0. \quad (1)$$

$$\delta(q_0, 1) \rightarrow \{q_1\}, \text{ that is, for input } 1, q_0 \text{ transits to } q_1. \quad (2)$$

$$\delta(q_1, 0) \rightarrow \phi, \text{ that is, for input } 0, \text{ there is no transition from } q_1. \quad (3)$$

$$\delta(q_1, 1) \rightarrow \{q_0, q_1\}, \text{ that is, } q_1 \text{ transits to both } q_0 \text{ and } q_1 \text{ for input } 1. \quad (4)$$

Step 2: In step 1, we have got a new state $\{q_0, q_1\}$. Now step 1 is repeated for this new state only, that is,

$$\delta(\{q_0, q_1\}, 0) \rightarrow \delta(q_0, 0) \cup \delta(q_1, 0) \quad (A)$$

$$\text{Since } \delta(q_0, 0) \rightarrow \{q_0, q_1\} \quad (\text{from equation (1)})$$

$$\text{And } \delta(q_1, 0) \rightarrow \phi \quad (\text{from equation (3)})$$

Therefore, equation (A) now becomes

$$\delta(\{q_0, q_1\}, 0) \rightarrow \delta(q_0, 0) \cup \delta(q_1, 0) \rightarrow \{q_0, q_1\} \cup \phi \rightarrow \{q_0, q_1\} \quad (5)$$

$$\text{Now, consider } \delta(\{q_0, q_1\}, 1) \rightarrow \delta(q_0, 1) \cup \delta(q_1, 1) \quad (B)$$

$$\text{Since } \delta(q_0, 1) \rightarrow \{q_1\} \quad (\text{from equation (2)})$$

$$\text{And } \delta(q_1, 1) \rightarrow \{q_0, q_1\} \quad (\text{from equation (4)})$$

Therefore, equation (B) now becomes

$$\delta(\{q_0, q_1\}, 1) \rightarrow \delta(q_0, 1) \cup \delta(q_1, 1) \rightarrow \{q_1\} \cup \{q_0, q_1\} \rightarrow \{q_0, q_1\} \quad (6)$$

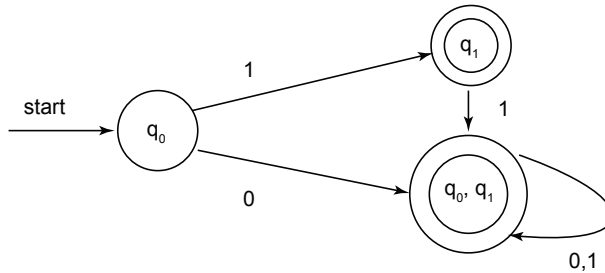
Now, based on equations (1)–(6), we will construct the following transition table:

States	Inputs	
	0	1
$\rightarrow \{q_0\}$	$\{q_0, q_1\}$	$\{q_1\}$
$\{q_1\}$	ϕ	$\{q_0, q_1\}$
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_1\}$

Since the starting state of given NFA is q_0 , it will also be the starting state for DFA. Moreover, q_1 is the final state of NFA; therefore, we have to consider all those set of states containing q_1 as the member. All such sets will become the final states of DFA. Thus, F for the resultant DFA is:

$$F = \{\{q_1\}, \{q_0, q_1\}\}$$

The equivalent DFA for the given NFA is as follows:



Now, we will relabel the DFA as follows:

$$q_0 \rightarrow A$$

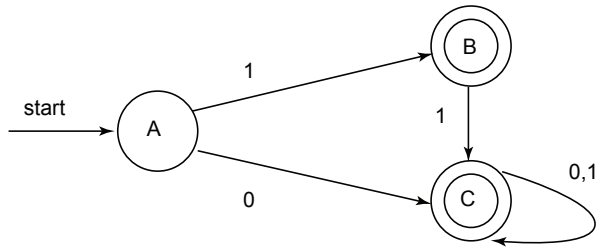
$$q_1 \rightarrow B$$

$$\{q_0, q_1\} \rightarrow C$$

The transition table now becomes

States	Inputs	
	0	1
$\rightarrow A$	C	B
$*B$	–	C
$*C$	C	C

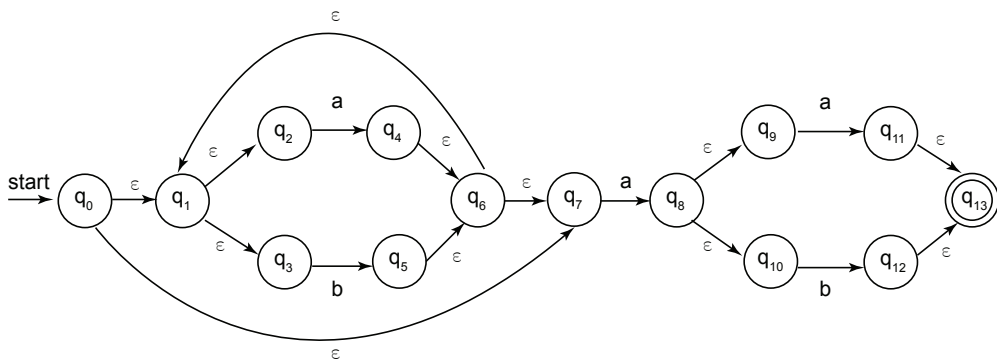
The equivalent DFA after relabeling is as follows:



26. Convert the given regular expression into NFA.

$(a/b)^* a (a/b)$

Ans:



Multiple-Choice Questions

- A _____ acts as an interface between the source program and the rest of the phases of compiler.
 - Semantic analyzer
 - Parser
 - Lexical analyzer
 - Syntax analyzer
- Which of these tasks are performed by the lexical analyzer?
 - Stripping out comments and whitespace
 - Correlating error messages with the source program
 - Performing the expansion of macros
 - All of these
- A _____ is any finite set of strings over some specific alphabet.
 - Sentence
 - Word
 - Language
 - Character class
- If zero or more symbols are removed from the end of any string s , a new string is obtained known as a _____ of string s .
 - Prefix
 - Suffix
 - Substring
 - Subsequence

5. If we have more than one possible transition on the same input symbol from some state, then the recognizer is said to be _____.
 - (a) Non-deterministic finite automata
 - (b) Deterministic finite automata
 - (c) Finite automata
 - (d) None of these
6. A tool for automatically generating a lexical analyzer for a language is defined as _____.
 - (a) Lex
 - (b) YACC
 - (c) Handler
 - (d) All of these
7. For `A = 10 to 50 do`, in the given code, A is defined as a/an _____.
 - (a) Constant
 - (b) Identifier
 - (c) Keyword
 - (d) Operator
8. The language for C identifiers can be described as: `letter_(letter_|digit)*`, here * indicates _____.
 - (a) Union
 - (b) Zero or more instances
 - (c) Group of subexpressions
 - (d) Intersection
9. The operation $P^* = \bigcup_{i=0}^{\infty} P^i$ represents
 - (a) Kleene closure of P
 - (b) Positive closure of P
 - (c) Concatenation
 - (d) None of these
10. A _____ is a compact notation that is used to represent the patterns corresponding to a token.
 - (a) Transition diagram
 - (b) Regular expression
 - (c) Alphabet
 - (d) Input buffer

Answers

1. (c) 2. (d) 3. (c) 4. (a) 5. (a) 6. (a) 7. (b) 8. (b) 9. (a) 10. (b)